

Hermetic WireGuard test harness — autonomous validation without live Mullvad

Revision: 2 Last modified: 2026-07-02T01:00:00Z Status: H0 DONE (real encrypted WireGuard tunnel round-trip proven), H1/H2/H3 next. Authority: inherits constitution/Constitution.md per §11.4.35. Serves the operator's deep-research mandate ("new ideas, game-changing approaches, opensource we can incorporate") + the §11.4.52 autonomous-validation mandate. **Rev 2 update:** the planned userspace-wireguard-go build proved unnecessary — the host wireguard **kernel module** is usable **inside** an unprivileged users netns (ip link add wg0 type wireguard + wg set succeed under unshare -Ur), so H0-full is a zero-build, zero-dependency, no-package-install path. Proven by tests/vpn_lan/hermetic_wg_roundtrip.sh: a real encrypted kernel-WireGuard tunnel between two unprivileged netns, HTTP payload round-tripped over the tunnel, sha256 + wg show handshake (latest-handshake=1782947082 rx=452 tx=752) verified, 3/3 deterministic (§11.4.50), with a wrong-peer-key golden-bad proving the crypto is load-bearing (rx=0 on a bad key, §11.4.107(10)/§11.4.68).

1. The problem this removes

The VPN-LAN feature ships 5 protocol round-trip tests (SMB/NFS, FTP/SFTP/WebDAV, email, Chromecast/DIAL, ADB) that are today **operator-gated SKIPS** — they cannot run without the operator's **live sword/Mullvad bridge** (secrets + admin) AND a working host podman. §11.4.52 (Autonomous-Validation) classifies an "only-validatable-with-the-operator-present" feature as a release blocker until promoted: the path does not scale to CI, does not run on every commit, and masks drift between manual runs.

The gap: we have never proven the *routing + proxy + protocol-client logic* works over an L3 tunnel **autonomously** — only that it honestly SKIPS when the bridge is down. That is correct anti-bluff behaviour (§11.4.3/§11.4.68), but it leaves the logic layer unproven except by the operator.

2. The approach — a loopback WireGuard pair in unprivileged network namespaces

Stand up **two network namespaces on one host, joined by a userspace-WireGuard tunnel**, entirely **rootless** and **without the real Mullvad bridge**:

```
netns A ("helix_proxy side")          netns B ("remote LAN
simulator")
10.9.0.1 — wg0 == encrypted L3 == wg0 — 10.9.0.2
| runs the routing/proxy path          | runs REAL
services:                               |
| + the protocol-client tests          | smbd · vsftpd · a
WebDAV http                             |
└──────────────────────────────────┘   · a DIAL/eureka
stub · mDNS
```

The tunnel is the **same L3 shape** as the real sword bridge (WireGuard + 10.0.0.0/8), so the identical routing/proxy code path is exercised. The protocol tests round-trip real bytes and assert sha256 / mailbox body / eureka JSON against the peer — the same anti-bluff assertions they already carry, now driven autonomously against a **controlled** peer.

Why it is rootless (the unlock)

`unshare -Ur -n` maps the caller's uid to **root inside a new user namespace**, so `ip netns add / ip link add ... type wireguard|veth` (normally root-only) succeed with **no real host privilege**, and everything created is confined to that throwaway namespace — invisible to every other process (§11.4.174-safe by construction). Kernel WireGuard inside a usersns needs `CAP_NET_ADMIN` *within that usersns* (granted by `unshare -Ur`); where the kernel module path is unavailable, **userspace WireGuard** (`wireguard-go / boringtun`) opens `/dev/net/tun` and needs no module at all (the `wireguard4netns` pattern below).

3. Feasibility — FACT-proven on THIS host (§11.4.6, captured probe 2026-07-02)

Check	Command	Result
unprivileged usersns clone enabled	<code>cat /proc/sys/kernel/unprivileged_usersns_clone</code>	1

Check	Command	Result
user-namespace budget	cat /proc/sys/user/max_user_namespaces	255793
net-namespace budget	cat /proc/sys/user/max_net_namespaces	255793
usersns+netns actually works	unshare -Ur -n sh -c 'ip link show lo && echo INNER_OK'	rc=0, INNER_OK
TUN device for userspace WG	test -c /dev/net/tun	present
unshare tool	unshare --version	util-linux 2.39.2

Conclusion (fact, not guess): a rootless, no-Mullvad, **podman-independent** hermetic WireGuard loopback harness is feasible on this host. It is gated on **neither** operator blocker (the broken host podman nor the live sword connection) — only on building the harness + a userspace-WG binary (both rootless, no host changes).

Missing (buildable, no root): wireguard-go / boringtun / wg are not installed; wireguard-go is a pure-Go go build (this is already a Go project) needing no privilege.

4. OSS to incorporate (§11.4.74 catalogue-first / extend-don't-reimplement)

Project	Role in the harness	Catalogue-Check
wireguard.com/netns	canonical two-netns WG routing pattern	reuse (reference)
davevel/wg-netns	declarative WG-in-netns profile setup	reuse or vendor the pattern
cmusatyalab/wireguard4netns	userspace WG (wireguard-go) into an unprivileged netns — the rootless path	reuse — this is the load-bearing dependency
rootless-containers/slirp4netns	user-mode networking for unprivileged netns (fallback egress)	reuse if a real-egress leg is ever needed

We do **not** reimplement WireGuard; we drive wireguard-go per the wireguard4netns pattern and script the netns lifecycle.

5. Per-protocol autonomy (honest — not every service is trivially rootless)

Protocol	Peer-B server	Rootless-feasible?
SMB/CIFS	smbd on a high port, private conf	yes (userspace, unprivileged)
FTP/FTPS	vsftpd / pure-ftpd high port	yes
WebDAV	nginx/apache/a Go WebDAV on a high port	yes
HTTP/DIAL (eureka)	a tiny Go :8008 eureka stub	yes
mDNS/DNS-SD	avahi/a Go mDNS responder	partial — avahi wants dbus; a Go responder avoids it
IMAP/SMTP/POP3	dovecot+a submission stub, or a Go mail stub	partial — real dovecot is heavy; a Go stub proves the client logic
NFS	kernel nfsd	no in usersns — use a userspace NFS (unfsd/Go NFS) or keep NFS operator-gated

The design promotes the **feasible** rows to AUTONOMOUS_VERIFIED and keeps the genuinely kernel-bound rows (NFS-kernel) as honest §11.4.3 operator-gated with the reason cited.

6. Honest boundary (§11.4.6 / §11.4.3 — what this does and does NOT prove)

- **Proves (autonomously):** the routing-over-an-L3-WG-tunnel path, the proxy path, the protocol-client round-trip logic (bytes/sha256/body/JSON) against a controlled peer, and that the anti-bluff teeth fire on a wrong answer — all with captured evidence per §11.4.5/§11.4.69/§11.4.107, deterministic per §11.4.50.
- **Does NOT prove:** the **real** remote topology — the actual Mullvad path, the real sword host's specific services, MTU/latency/firewall quirks of the production peer. That remains a §11.4.3 **real-topology confirmation** run, still operator-gated on the live bridge. The harness converts "can't test at all without you" → "logic autonomously proven; real-topology confirmation operator-gated" — a large §11.4.52 promotion, not a replacement for the live run.

7. Bridge-contract integration (§11.4.28 — no coupling)

Add a `HELIX_BRIDGE_MODE=hermetic` to `tests/lib/sword_bridge.sh` consumers: in hermetic mode the harness sets `HELIX_BRIDGE_HEALTH` to the loopback peer's reachability probe and `HELIX_BRIDGE_SUBNET/HELIX_BRIDGE_HOST` to the peer-B `10.9.0.0/24 / 10.9.0.2`. The existing tests are **unchanged** — `bridge_require` simply returns 0 (up) against the hermetic peer instead of SKIP, so the same assertions run. Default/unset mode = today's behaviour (real bridge or honest SKIP). No test learns it is talking to a loopback peer.

8. Phased plan (each phase = one PWU, §11.4.58; all rootless, non-operator-gated)

1. **H0 — DONE** (f96da56 veth substrate + this-round kernel-WG tunnel). The planned wireguard-go build was unnecessary: the host wireguard **kernel module** works inside an unprivileged users netns. `tests/vpn_lan/hermetic_netns_poc.sh` (veth L3 round-trip) + `tests/vpn_lan/hermetic_wg_roundtrip.sh` (real encrypted WireGuard tunnel) both PASS deterministically with golden-bad teeth; physical proof (`wg show handshake + rx/tx, sha256`) under `qa-results/vpn_lan/hermetic{,_wg}/`. Teardown is automatic — the whole namespace dies with `unshare` (§11.4.14).
2. **H1** — peer-B service launcher (`smbd/vsftpd/webdav/eureka/mDNS` Go responder, high ports, private configs, no root). Physical proof: each service reachable from peer-A over the tunnel.
3. **H2** — wire `HELIX_BRIDGE_MODE=hermetic` + run the existing protocol tests against the peer; promote the feasible rows to `AUTONOMOUS_VERIFIED` in the §11.4.52 ledger; NFS-kernel stays honest operator-gated. Each is a §11.4.135 standing guard.
4. **H3** — a §1.1 mutation per protocol proving the hermetic assertions are load-bearing (wrong sha256 in peer-B ⇒ test FAILs), and a determinism pass (§11.4.50, N iterations).

9. Host caveat (observed 2026-07-02)

During feasibility probing the host briefly hit **user-process/thread exhaustion** (fork: retry: Resource temporarily unavailable, `errno=11`) — the same accumulated subagent-transcript + broken-podman-container detritus that fills `/tmp`. The harness MUST be **process-frugal** (a bounded, fixed number of processes, all reaped on teardown) and MUST run only when the

host has process headroom — a pre-flight `ulimit -u` / live-process check that honestly SKIPS (§11.4.3) rather than adding pressure to a starved host (§12 / §11.4.133).

10. Composition

Composes §11.4.3 / §11.4.6 / §11.4.8 / §11.4.28 / §11.4.50 / §11.4.52 / §11.4.68 / §11.4.69 / §11.4.74 / §11.4.107 / §11.4.135 / §11.4.150 / §11.4.161 / §11.4.174 / §12.

Sources verified 2026-07-02

- <https://www.wireguard.com/netns/> — canonical WireGuard network-namespace routing pattern (root-based reference).
- <https://github.com/dadevel/wg-netns> — declarative WireGuard-in-netns profiles, multi-peer.
- <https://github.com/cmusatyalab/wireguard4netns> — userspace WireGuard (wireguard-go) into an **unprivileged** network namespace (the rootless load-bearing path).
- <https://github.com/rootless-containers/slirp4netns> — user-mode networking for unprivileged network namespaces.
- <https://www.procustodibus.com/blog/2022/10/wireguard-in-podman/> — WireGuard in rootless Podman (NET_ADMIN/NET_RAW, network_mode: service: pattern).